Today's Lab — Four Programs on Regression & Classification

Program 1

Linear Regression on California Housing Dataset

Build a linear regression model on the California Housing dataset. Learn feature scaling, train-test split, and evaluate using MSE, RMSE, and R^2 score.

Program 2

Logistic Regression for Binary Classification

Use logistic regression on the Iris or Breast Cancer dataset. Plot the confusion matrix, ROC curve, and visualize the decision boundary.

Program 3

MNIST Digit Classification using Multiple Classifiers

Load the MNIST handwritten digits dataset and compare Logistic Regression, SVM, and Random Forest classifiers using accuracy, precision, recall, and F1-score.

Program 4

Multiclass Classification — SVM, Random Forest & Neural Networks

Apply and compare three classifiers on the Iris or Wine dataset. Measure accuracy, macro F1-score, and training time; discuss trade-offs.

Datasets You Will Use for Programs 1 – 4

California Housing Dataset (Program 1)

```
from sklearn.datasets import
fetch_california_housing
```

20,640 samples, 8 features; target: median house value

docs: scikit-learn.org/stable/datasets/real_world.html

Iris / Breast Cancer Dataset (Program 2)

```
from sklearn.datasets import load_iris
from sklearn.datasets import load_breast_cancer
```

UCI: dataset/53/iris | dataset/17

MNIST Handwritten Digits (Program 3)

```
from keras.datasets import mnist
  or fetch_openml('mnist_784')
```

yann.lecun.com/exdb/mnist

Iris / Wine Dataset (Program 4)

```
from sklearn.datasets import load_iris
from sklearn.datasets import load_wine
```

UCI Wine: dataset/109/wine

Quick Install: `pip install scikit-learn keras tensorflow pandas numpy matplotlib seaborn`

Program 1 — Linear Regression on California Housing Dataset

Program 1

Task: Load the California Housing dataset and build a linear regression model to predict median house value (MedHouseVal) using features like median income, house age, and average rooms. Split 80% training / 20% testing. Print MSE, RMSE, and R^2 score. Plot actual vs predicted values.

Expected Output:

```
Feature matrix shape : (20640, 8)
Training MSE     : 0.518
Testing MSE      : 0.556
Testing RMSE     : 0.745
Testing R²       : 0.596
[Scatter plot: Actual vs Predicted saved]
```

Steps to implement:

1. Load California Housing via `fetch_california_housing`; separate features X and target y
2. Standardize features with `StandardScaler`; split 80/20 with `train_test_split`
3. Fit `LinearRegression` on training data; predict on both train and test sets
4. Compute MSE, RMSE, and R^2 using `mean_squared_error` and `r2_score`
5. Plot Actual vs Predicted scatter with `matplotlib`; save the figure

Program 2 — Logistic Regression for Binary Classification

Program 2

Task: Use the Breast Cancer dataset to train a binary logistic regression classifier. Split 70% train / 30% test (stratified). Compute confusion matrix, accuracy, precision, recall, F1-score, and ROC-AUC score. Plot the ROC curve.

Expected Output:

```
Confusion Matrix:
[[TN  FP]    True Negatives : 107
[FN  TP]]  True Positives : 62
Accuracy  : 94.15%
Precision : 0.961  Recall : 0.924
F1-Score  : 0.942  ROC-AUC: 0.989
[ROC curve plot saved]
```

Steps to implement:

1. Load Breast Cancer dataset; check class distribution; select features X and binary target y
2. Standardize features; perform stratified 70/30 split using `train_test_split(stratify=y)`
3. Fit `LogisticRegression`; predict class labels and probabilities on the test set
4. Build confusion matrix; compute accuracy, precision, recall, F1-score, and ROC-AUC
5. Compute `fpr, tpr` with `roc_curve`; plot ROC curve and save the figure

Program 3 — MNIST Digit Classification (3 Classifiers)

Program 3

Task: Load the MNIST dataset (28x28 grayscale images, digits 0–9). Train three classifiers: Logistic Regression, SVM (RBF kernel), and Random Forest. Compare all three using accuracy, precision, recall, and F1-score on the 10,000-sample test set.

Expected Output:

```
Train: 60000 samples  Test: 10000 samples
Classifier          Accuracy  F1-score
Logistic Regression  92.46%   0.924
SVM (kernel=rbf)     97.18%   0.972
Random Forest        96.82%   0.968
[Bar chart: Accuracy comparison saved]
```

Steps to implement:

1. Load MNIST via `keras.datasets.mnist`; normalize pixels to [0,1] by dividing by 255.0
2. Flatten 28×28 images into 784-dim vectors; subsample for SVM speed
3. Train LogReg, `SVC(kernel='rbf')`, and `RandomForestClassifier`
4. Predict on test set; compute accuracy, precision, recall, F1 (`average='macro'`)
5. Plot a bar chart comparing the three classifiers' accuracy; save the figure

Program 4 — Multiclass: SVM, Random Forest & MLP

Program 4

Task: Use the Wine dataset (3 classes, 13 features). Train SVM, Random Forest, and `MLPClassifier` (Neural Network). Evaluate accuracy and macro F1-score for each. Measure and compare training time. Discuss accuracy vs speed trade-offs.

Expected Output:

```
Dataset: 178 samples, 13 features, 3 classes
Classifier      Accuracy  F1(macro)  Time(s)
SVM (rbf)       98.33%    0.983      0.12
Random Forest   97.78%    0.978      0.08
MLPClassifier   96.67%    0.965      2.45
[Comparison bar chart saved]
```

Steps to implement:

1. Load Wine dataset from sklearn; standardize features; split 75% train / 25% test
2. Initialize `SVC(kernel='rbf')`, `RandomForestClassifier`, and `MLPClassifier`
3. Train each model; record training time using `time.time()` before/after `.fit()`
4. Predict on test set; compute accuracy and macro F1-score with `average='macro'`
5. Plot grouped bar chart of accuracy / F1 / time; discuss accuracy vs speed trade-offs